

Projet : Gestionnaire de tâches collaboratif

Plan détaillé + Estimations

Étape 1 — Analyse & Conception

Objectif : définir précisément le fonctionnement du projet avant d'écrire du code.

♦ **Tâches :**

- Définir les **fonctionnalités principales** (création de projet, ajout de tâches, suivi, partages)
- Identifier les **utilisateurs types** (membres, admin, invité ?)
- Rédiger un mini **cahier des charges fonctionnel**
- Dessiner un **diagramme de cas d'utilisation (UML)**
- Faire un **diagramme de classes (UML)** pour les entités principales
- Concevoir les **routes de l'API REST** (endpoints, méthodes, données attendues)
- Esquisser des **maquettes simples de l'interface** (web + Qt, même au crayon)



Estimation : 6 à 10 heures

Étape 2 — Développement du back-end (API REST)

Objectif : créer le moteur de ton application.

♦ **Tâches :**

- Initialiser le projet back-end avec **Flask, FastAPI** (Python) ou **Spring Boot** (Java)
- Définir les **routes principales** : /users, /tasks, /projects
- Implémenter :
 - Ajout / modification / suppression de tâches
 - Assignment de tâches à un utilisateur
 - Connexion / inscription
- Gérer la **base de données** (SQLite, PostgreSQL...)
- Implémenter une **authentification** basique (token ou session)
- Ajouter **CORS** pour le front-end
- Tester les routes avec **Postman** ou **curl**

 **Estimation : 15 à 25 heures**

Étape 3 — Développement du front-end Web

Objectif : créer une interface accessible via un navigateur.

♦ **Tâches :**

- Créer une structure HTML/CSS avec TypeScript
- Implémenter les pages :
 - Connexion / inscription
 - Tableau de bord
 - Liste des tâches
 - Formulaire de création/édition
- Utiliser **fetch()** ou **Axios** pour communiquer avec l'API
- Afficher les données dynamiquement (DOM, rendering conditionnel)
- Gérer les erreurs et les chargements
- Ajouter du style (CSS ou Tailwind)

 **Estimation : 15 à 20 heures**

Étape 4 — Application Desktop (Qt en C++)

Objectif : proposer une version native du gestionnaire de tâches.

♦ **Tâches :**

- Créer l'interface principale avec **Qt Designer** ou en code :
 - Liste de tâches
 - Formulaire d'ajout/édition
 - Détails d'une tâche
- Communiquer avec l'API via **requêtes HTTP en C++**
- Gérer :
 - La sérialisation JSON
 - Les erreurs réseau
 - L'authentification
- Ajouter un mode **hors ligne** (lecture/écriture locale)

 **Estimation : 20 à 30 heures**

Étape 5 — Synchronisation & fonctionnalités avancées

Objectif : rendre l'expérience utilisateur plus fluide et complète.

♦ **Tâches :**

- Ajouter une synchronisation automatique (API ↔ local)
- Gérer les **conflits de données**
- Ajouter des fonctionnalités optionnelles :
 - Notifications
 - Fichiers joints
 - Filtrage des tâches (par utilisateur, par date)
 - Dark mode ?

 **Estimation : 8 à 12 heures**

Étape 6 — Tests, validation et déploiement

Objectif : fiabiliser et publier ton projet.

♦ **Tâches :**

- Tester toutes les fonctionnalités manuellement
- Écrire quelques **tests automatiques** (si tu veux)
- Corriger les bugs
- Déployer l'API (ex. : Render, Fly.io, Railway)
- Déployer le front-end (GitHub Pages, Vercel...)
- (Optionnel) Créer un installeur pour l'application Qt

 **Estimation : 5 à 10 heures**

Résumé total du projet

Phase	Durée estimée
 Analyse & Conception	6 – 10 h
 Back-end API	15 – 25 h
 Front-end Web	15 – 20 h

Phase	Durée estimée
 Application Qt	20 – 30 h
 Synchronisation & Bonus	8 – 12 h
 Tests et déploiement	5 – 10 h
 Total estimé	69 – 107 h

Conseils pratiques

- Note ton temps passé par phase pour suivre ton avancement
- Travaille **par livrables** (ex : back-end testé → check )
- Tu peux aussi faire un **journal de bord** si tu veux valoriser le projet (portfolio, oral, etc.)